arXiv:2605.10763v1 [cs.AI] 11 May 2026

MATRA: Modeling the Attack Surface of Agentic AI Systems - OpenClaw Case Study

Tim Van hamme, Thomas Vissers, Javier Carnerero-Cano, Mario Fritz, Emil C. Lupu, Lieven Desmet, Dinil Mon Divakaran*†

Abstract

LLMs are increasingly deployed as autonomous agents with access to tools, databases, and external services, yet practitioners (across different sectors) lack systematic methods to assess how known threat classes translate into concrete risks within a specific agentic deployment. We present MATRA, a pragmatic threat modeling framework for agentic AI systems that adapts established risk assessment methodology to systematically assess how known LLM threats translate into deployment-specific risks. MATRA begins with an asset-based impact assessment and utilizes attack trees to determine the likelihood of these impacts occurring within the system architecture. We demonstrate MATRA on a personal AI agent deployment using OpenClaw, quantifying how architectural controls such as network sandboxing and least-privilege access reduce risk by limiting the blast radius of successful injections.

*Tim Van hamme, Thomas Vissers and Lieven Desmet are with DistriNet, KU Leuven, 3001 Leuven, Belgium ({tim.vanhamme, thomas.vissers, lieven.desmet}@kuleuven.be), Javier Carnerero-Cano with IBM Research (javier.cano@ibm.com), Mario Fritz with CISPA Helmholtz Center for Information Security (fritz@cispa.de), Emil C. Lupu with Imperial College London (e.c.lupu@imperial.ac.uk) and Dinil Mon Divakaran with A*STAR Institute for Infocomm Research (dinil_divakaran@a-star.edu.sg).

†Accepted for presentation at the 5th International Workshop on Designing and Measuring Security in Systems with AI (DeMeSSAI 2026), co-located with the 11th IEEE European Symposium on Security and Privacy (EuroS&P 2026), Lisbon, Portugal, July 10, 2026.

1 Introduction

Large Language Models (LLMs) are increasingly being embedded into enterprise software systems that ingest untrusted content and produce operational decisions, recommendations, or even executable plans. In these *LLM-integrated* applications, security failures are increasingly driven not only by the behavior of the model in isolation, but by how model outputs are connected to, e.g., privileged tools, databases, retrieval pipelines, and downstream services [1].

Recently, researchers have demonstrated a growing catalogue of LLM safety and security failures, including jailbreaking and prompt injection, and the ecosystem has responded with guardrails [2, 3, 4], automated red-teaming methods [5, 6, 7], and benchmarks for attacks and defenses [8, 9]. Yet, a persistent disconnect remains between vulnerability-centric evaluations and the concrete risks faced by enterprises deploying LLMs in real business workflows, where harm is realized only after a model failure propagates through application logic and connected assets [10, 11].

Prompt injection is now widely recognized as a practical and difficult-to-mitigate attack [12] in LLM-integrated systems. Beyond direct user-to-model injections, *indirect* prompt injection can be delivered via retrieved documents or tool outputs, allowing adversaries to influence an application without a direct interface [13]. This risk is amplified in *agentic* workflows, where an LLM iteratively plans and executes tool calls: agentic pipelines blur the boundary between *instructions* and *data* [14], making agents susceptible to being steered by untrusted contents they

process as part of the task. Recent benchmarks show that attacks can succeed at multiple stages of the agentic loop and existing defenses provide limited and inconsistent protection [12, 15, 16].

A fundamental constraint underlies these threats: prompt injection is functionally equivalent to delivering new instructions to the LLM, much as code injection delivers executable instructions to a computer. Existing defenses can reduce the probability of successful injection or potentially detect the injection, but cannot eliminate it [12, 17, 18]. Moreover, the attack surface extends beyond a single prompt-response interaction—persistent components such as system prompts, long-term memory, and retrieval-augmented generation (RAG) knowledge bases can be targeted for long-horizon influence and cross-session compromise [19].

In response, the community has produced valuable taxonomies and governance guidance for GenAI risks, including the OWASP Top 10 for LLM Applications [20], the OWASP Agentic AI Threats and Mitigations guide [21], as well as NIST's profile for generative AI risks [22]. However, these resources primarily describe *what* classes of issues exist. Enterprises still lack a systematic framework to (i) ground threats in a specific deployed workflow, (ii) trace paths from impact scenarios to concrete attack vectors across tools, memory, and retrieval components, and (iii) prioritize mitigations based on business impact.

We address this gap with `MATRA`[1](Modeling Attacks and Threats for Risk Assessment), an impact-driven threat and risk modeling framework for agentic AI systems. `MATRA` is inspired by NIST SP 800-30 risk assessment concepts [23] and models system risks by constructing attack trees that connect CIA-based impact scenarios to attacker objectives, techniques, and architecture-specific vectors. `MATRA` captures not only on adversarial manipulation but also non-adversarial pathways in which unexpected LLM behavior combines with autonomous tool execution to produce high-impact outcomes.

We demonstrate `MATRA` on OpenClaw, an open-source *single-agent, multi-tool* personal assistant architecture that integrates messaging channels, high-privilege tools (shell, file I/O, web), and persistent memory [24]. Our case study shows how architectural patterns determine the feasible attack surface and how controls can be evaluated for risk reduction from two complementary perspectives: (i) the weakest-link (easiest-path) risk; and (ii) the full-attack surface exposure across feasible vectors.

[1] Mātrā in Sankskrit denotes measure, quantify, etc.

2 `MATRA`: Threat and Risk Modeling Framework

`MATRA` framework is inspired by NIST Special Publication 800-30, "Guide for Conducting Risk Assessments" [23]. We demonstrate it using a **semi-quantitative approach** to risk assessment, where structured scales translate expert judgment into ratings that enable relative risk comparison and prioritization while remaining accessible to decision-makers. Other assessment approaches, e.g., fully quantitative or purely qualitative, can be used within `MATRA`.

The assessment is **impact-oriented**: it begins with the identification of critical assets and the business impacts of concern, then traces backward to the threat events and sources that could cause those impacts. This ensures that vulnerabilities and adversarial actions are examined in the context of their potential to create unacceptable outcomes rather than in isolation. Figure 1 summarizes the framework's information flow. A practitioner proceeds through four stages: system analysis, impact assessment, attack tree construction, and risk quantification.

2.1 System Analysis

The assessment begins by collecting system properties from the stakeholder: architecture documentation, source code, deployment configurations, and any existing security documentation. From these inputs, the practitioner constructs a **data flow diagram (DFD)** that maps processes, data stores, external actors, and trust boundaries.

The DFD serves two purposes. First, it identifies the system's **assets**, i.e., the critical resources requiring protection, such as databases, credentials, user

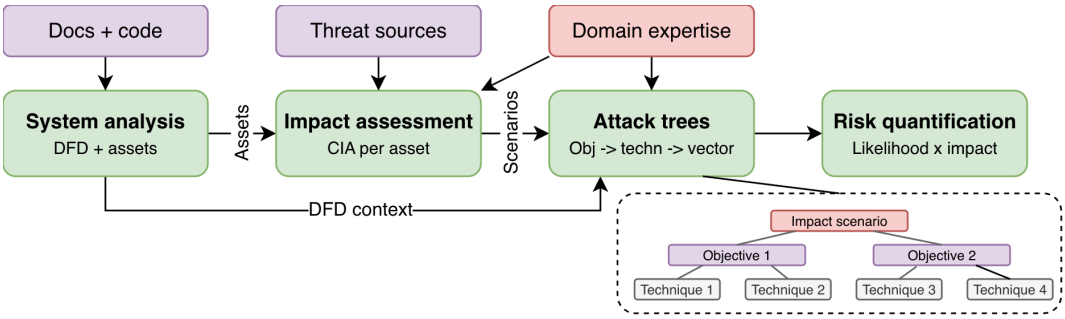


Figure 1: `MATRA` framework overview. System properties and threat sources are collected from the client. Assets identified from system documentation feed into a stakeholder-driven business impact assessment, which produces impact scenarios. A data flow diagram (DFD), combined with known attack techniques from established catalogs, informs the construction of attack trees that decompose each impact scenario into objectives, techniques, and architecture-specific vectors.

data, and integration interfaces. Systematically identifying assets is a prerequisite for the impact assessment that follows, as it enables the direct mapping of potential impact scenarios to specific system components. Second, the DFD exposes the system's communication paths and trust boundaries, which later inform the identification of concrete attack vectors.

2.2 Impact Assessment

With assets identified, the practitioner selects the **threat sources** relevant to the deployment. Threat sources are entities or conditions capable of exploiting vulnerabilities within the system; ranging from nation-state actors and competitors to insiders and accidental causes (see Appendix B for the full taxonomy adapted from NIST SP 800-30 [23]). Only threat sources with credible motivation and a viable access path to the system under review are retained for analysis; this scoping step filters out actors that lack intent or a realistic targeting path.

For each retained threat source, or per threat source category when impact is independent of the attacker's identity, the stakeholder assesses **business impact** across the CIA triad—confidentiality, integrity, and availability—for every asset in scope:

"What is the business impact if this asset's [confidentiality / integrity / availability] is compromised by [threat source]?"

Impact is rated on a three-grade scale: **Low**: Disruption, financial loss, or reputation harm is limited. Mitigation is quick and recovery is straightforward. **Moderate**: Disruption or losses are noticeable. Recovery requires effort and additional resources. **High**: Major disruption of operations, significant financial losses or reputation harm. Recovery is prolonged or potentially impossible.

From this matrix, the practitioner derives **impact scenarios** that are distinct, architecturally viable adverse outcomes, rated at the highest assessed level across threat sources. Each impact scenario names a specific asset, a CIA dimension, and the concrete harm (e.g., "customer PII exfiltrated from the production database").

2.3 Attack Tree Construction

To connect impact scenarios to concrete threats, the practitioner builds **attack trees** [25]—hierarchical diagrams that decompose high-level impacts into controllable, architecture-specific attack paths. The nodes of an attack tree are:

- **Root:** The *impact scenario*. Focusing on the most consequential impacts helps manage scale.
- **Attack Objectives:** One or more high-level

goals that must be achieved to realise the impact (e.g., LLM behavioural manipulation combined with data exfiltration).

- **Attack Techniques:** Methods to achieve an objective such as indirect prompt injection.
- **Attack Vectors:** Concrete instances of a technique in the system architecture, e.g., indirect prompt injection via a support email.

The practitioner constructs trees in three stages. First, each impact scenario is decomposed into one or more attack objectives. Second, well-established LLM-specific threat repositories from, say, OWASP [20, 21, 26], MITRE [27], NIST [22] CSA [28], and Cisco [29] are consulted alongside the DFD and system model to select the most relevant attack techniques and to identify existing security controls. Third, each technique is mapped to the concrete attack vectors present in the application architecture, informed by the different paths in DFD and the trust boundaries.

This method ensures that each attack tree reflects both recognised industry threats and the specific architecture of the system under review.

2.4 Risk Quantification

For each attack vector, likelihood is derived in two steps. First, *capability fit* compares the threat source's technical capability with the skill level required for the vector, using Table 1 (Appendix A); because the scoping step in Section 2.2 already filters for intent and targeting, capability remains the discriminating factor at the vector level. For non-adversarial threats, capability does not apply; instead, each vector receives an *inherent manifestation likelihood* reflecting the probability of the event occurring independently of any actor (e.g., hallucination probability). Second, *residual success likelihood* estimates the probability that the vector succeeds given existing security controls; a vector with no applicable controls receives a high residual likelihood, while effective controls reduce it. The overall vector likelihood is obtained by combining these two values via the combination matrix (Table 2, Appendix A).

Within each objective the most favourable (highest) vector likelihood is chosen, reflecting that an attacker will pursue the path of least resistance. Across objectives the overall scenario likelihood is the weakest link, i.e., the minimum of those maxima, because every objective must succeed for the impact to occur.

An impact scenario's risk is Risk = Likelihood $\times$ Impact, where qualitative levels (low, moderate, high) combine into a final rating from very low to very high via Table 3 (Appendix A). We draw mitigation decisions from two complementary perspectives: (i) *weakest-link risk*, which measures the single easiest attack path an adversary is most likely to exploit first (or that is most likely to manifest non-adversarially); and (ii) *full attack surface*, which captures total exposure across all feasible vectors, revealing cases where multiple comparable paths create material risk even if no single path dominates. Using both perspectives ensures that defences address not only the most obvious entry point but also the full spectrum of exploitable opportunities.

3 Use Case: Personal AI Agent

We demonstrate `MATRA` on OpenClaw, an open-source personal AI agent that follows a *single-agent, multi-tool* architecture: one LLM agent with simultaneous access to shell execution, file I/O, web browsing, cross-channel messaging, and persistent memory [24]. We model a solo developer using OpenClaw as a productivity tool for their SaaS booking platform, with an email plugin for customer support. The full deployment scenario, data flow diagrams, asset inventory, threat source selection, and complete attack trees are provided in Appendix C.

From the full assessment, we select one threat source–impact combination to illustrate `MATRA`'s end-to-end process: a *malicious customer* (threat source) targeting the *confidentiality of customer PII* (business impact). This combination is compelling because the threat source has a natural, legitimate access channel, i.e., customer support email, and requires no technical sophistication beyond crafting a convincing message with embedded instructions.

3.1 Deep Dive: Malicious Customer × Customer PII Confidentiality

In this scenario (IS6 in Table 5), a malicious customer exploits the agent's email channel to exfiltrate other customers' PII (personally identifiable information) and booking records, following the indirect prompt injection pattern demonstrated by Zhao et al. [30]. Two objectives must jointly succeed: *LLM behavioral manipulation* (redirecting the agent's actions) and *data exfiltration* (transmitting customer records outside the trust boundary).

The attacker sends a support email with hidden instructions embedded in an attachment (e.g., via HTML comments or zero-width Unicode characters). When the agent processes the email, the injected instructions direct it to query customer records via the `exec` tool and exfiltrate the results through tool-based channels (`curl`, email) or rendered-markup channels (embedding data in a markdown image URL that the Telegram client auto-fetches). The entire chain executes within a single agent turn.

Attack Tree with Likelihood Scores We assess this scenario under two control configurations: (a) default unsandboxed deployment, and (b) Docker sandboxing enabled (`network=none`). The resulting attack tree is shown in Figure 2.

Risk Assessment Impact is rated High due to the regulatory implications of customer PII disclosure. Without sandboxing, the scenario reaches Very High risk (9). Docker sandboxing with `network=none` blocks `exec`-based exfiltration (`curl`) but does not reduce the overall exfiltration objective: the message tool runs on the gateway host outside the container, so email-based exfiltration remains viable, and the agent's Telegram response can embed queried data in a markdown image URL that the client auto-fetches (zero-click). The scenario therefore remains at Very High (9) even with sandboxing.

Reducing IS6 to Moderate (3) requires controls that address the surviving exfiltration vectors: an email allow-list that restricts outbound recipients, and output sanitisation that strips or rewrites URLs containing encoded data from agent responses before they reach the messaging channel. With both controls in place, all exfiltration vectors are reduced to Low, yielding Moderate risk. The injection objective remains at High regardless—customer emails are *expected* input that the agent is designed to process.

3.2 Key Observations

Prompt Injection and Architectural Defense Prompt injection is functionally equivalent to delivering new instructions to the LLM: like code injection in traditional systems, the injected payload executes with whatever capabilities the host process provides. Because current defenses can reduce but not eliminate the probability of successful injection [12], the impact of an attack is ultimately determined by what the agent can *do* with the tools, integrations, and permissions available to it.

This has a direct consequence for risk mitigation. Rather than relying primarily on input-level defenses (prompt injection detectors, guardrails), the most effective controls are *architectural*: sandboxing that limits network access, allow-listing that restricts which tools and commands the agent may invoke, and least-privilege configurations such as read-only database roles. The IS8 analysis (Appendix C.8) demonstrates this clearly: a read-only database role eliminates the action execution objective entirely, reducing risk from High (6) to Moderate (3). However, the IS6 analysis reveals that no single architectural control is sufficient when multiple exfiltration channels exist—container sandboxing blocks `exec`-based vectors but leaves message-based and rendered-markup vectors unaffected. Effective defense requires *layered* architectural controls that address each exfiltration category.

Non-Adversarial Threats Autonomous tool execution creates a qualitatively new risk category: high-impact scenarios requiring no attacker. As demonstrated in the IS8 analysis (Appendix C.8), a hallucinated `UPDATE` with a missing predicate can corrupt customer booking records, trigger erroneous refunds, and produce incorrect reports—all from a routine developer request. In traditional systems, integrity impacts of this magnitude typically require adversarial

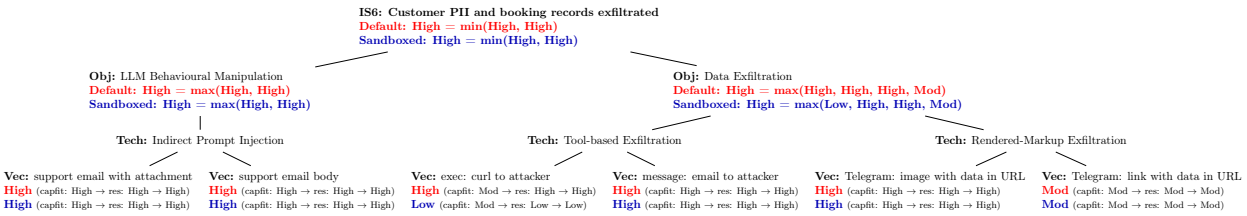


Figure 2: Attack tree for IS6 (customer data exfiltration; refer Table 5), assessed for threat source: *malicious customer* (capability: Low). Likelihood scores shown under default configuration (red) and with Docker sandboxing (blue). Vector labels follow the format *capfit: capability fit* → *res: residual success likelihood* → *combined likelihood* (see Tables 1 and 2). Sandboxing blocks `exec`-based network access (res: Low for curl) but does not affect message-based or rendered-markup vectors, which operate outside the container (res: High); the link vector's requirement for a user click partially reduces success likelihood (res: Mod).

action. In agentic systems, the LLM's imperfect reasoning combined with unrestricted tool access means that normal usage can trigger destructive operations. This has implications for risk frameworks: *accidental* threat sources must be assessed with the same rigor as adversarial ones when the system has autonomous execution capabilities.

4 Conclusions

We presented `MATRA`, an impact-driven threat and risk modeling framework for agentic AI systems. `MATRA` provides a structured process comprising system analysis, stakeholder-driven impact assessment, attack tree construction informed by established threat catalogs, and risk quantification. They connect high-level business impacts to architecture-specific attack vectors. Inspired by NIST SP 800-30 and grounded in data flow analysis, `MATRA` enables practitioners to move beyond generic vulnerability lists toward deployment-specific risk prioritization.

Our case study on OpenClaw reinforces a central insight: because prompt injection cannot be eliminated [12], the most effective risk reduction comes from limiting what injected instructions can achieve through layered architectural controls via dedicated planner and executer LLMs [31, 32], sandboxing, allow-listing, output sanitisation, and least-privilege configurations, each addressing different vector classes.

Several limitations point to future work. Attack tree completeness is bounded by an analyst's knowledge and the threat catalogs consulted. However, the hierarchical structure makes gaps localizable; when a new technique emerges, the affected objectives and impact scenarios are immediately identifiable. `MATRA` currently relies on expert judgment for likelihood assignment; we bound this subjectivity through deliberately coarse scales and auditable per-node ratings, but integrating empirical data from red-teaming benchmarks or operational logs could improve calibration. Furthermore, the min-max composition heuristic may underestimate risk when multiple moderate paths interact; the full attack surface perspective partially addresses this, but formal composition under uncertainty is a future direction. `MATRA` has been demonstrated on a single-agent architecture; applying it to multi-agent orchestrations with inter-agent communication would test its scalability and may require extensions to model cross-agent propagation paths. Finally, our assessment does not address privacy-specific threat modeling (e.g., LINDDUN[33]), which is an important complementary dimension for systems processing personal data.

Acknowledgments This research is partially funded by the Internal Funds KU Leuven, and by the Cybersecurity Research Program Flanders.

This research is also supported by the National

Research Foundation, Singapore, and the Cyber Security Agency of Singapore under the National Cybersecurity R&D Programme (NCR25-CRPO-0001) and the CyberSG R&D Programme Office (Award CRPO-GC5-ASTAR-001). Any opinions, findings, conclusions, or recommendations expressed in these materials are those of the author(s) and do not reflect the views of the National Research Foundation, Singapore, the Cyber Security Agency of Singapore, or the CyberSG R&D Programme Office.
Last but not the least, the authors thank Schloss Dagstuhl – Leibniz Center for Informatics for hosting Seminar 25461 on "Security and Privacy of Large Language Models,"[2] and its organizers and participants for stimulating discussions that sharpened this work.

References

[1] Meta AI, "Agents rule of two: A practical approach to AI agent security." Meta AI Blog, Oct. 2025. Accessed: 2026-03-16.

[2] ProtectAI.com, "Fine-tuned DeBERTa-v3-base for prompt injection detection." Hugging Face Model Hub, 2024.

[3] Google, "Model armor overview." Google Cloud Documentation, 2025.

[4] S. Chennabasappa, C. Nikolaidis, D. Song, D. Molnar, S. Ding, S. Wan, S. Whitman, L. Deason, N. Doucette, A. Montilla, A. Gampa, B. de Paola, D. Gabi, J. Crnkovich, J.-C. Testud, K. He, R. Chaturvedi, W. Zhou, and J. Saxe, "LlamaFirewall: An open source guardrail system for building secure AI agents." arXiv preprint, May 2025.

[5] A. Mehrotra, M. Zampetakis, P. Kassianik, B. Nelson, H. S. Anderson, Y. Singer, and A. Karbasi, "Tree of attacks: Jailbreaking black-box LLMs automatically," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.

[2] https://www.dagstuhl.de/25461

[6] L. Derczynski, E. Galinkin, J. Martin, S. Majumdar, and N. Inie, "garak: A framework for security probing large language models," 2024.

[7] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson, "Universal and transferable adversarial attacks on aligned language models," 2023.

[8] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.

[9] P. Chao, E. Debenedetti, A. Robey, M. Andriushchenko, F. Croce, V. Sehwag, E. Dobriban, N. Flammarion, G. J. Pappas, F. Tramèr, H. Hassani, and E. Wong, "Jailbreakbench: An open robustness benchmark for jailbreaking large language models," in *NeurIPS Datasets and Benchmarks Track*, 2024.

[10] Hack The Box, "Inside CVE-2025-32711 (EchoLeak): Prompt injection meets AI exfiltration." Hack The Box Blog, 2025. Accessed: 2026-03-16.

[11] S. Willison, "The summer of johann: Prompt injections as far as the eye can see." Simon Willison's Weblog, Aug. 2025. Part of the series *Prompt Injection*. Accessed: 2026-03-16.

[12] M. Nasr, N. Carlini, C. Sitawarin, S. V. Schulhoff, J. Hayes, M. Ilie, J. Pluto, S. Song, H. Chaudhari, I. Shumailov, A. Thakurta, K. Y. Xiao, A. Terzis, and F. Tramèr, "The Attacker Moves Second: Stronger Adaptive Attacks Bypass Defenses Against LLM Jailbreaks and Prompt Injections." `https://arxiv.org/abs/2510.09023`, 2025.

[13] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection," in *Proceedings of the 16th ACM*

Workshop on Artificial Intelligence and Security, AISec '23, (New York, NY, USA), p. 79–90, Association for Computing Machinery, 2023.

[14] E. Zverev, S. Abdelnabi, S. Tabesh, M. Fritz, and C. H. Lampert, "Can llms separate instructions from data? and what do we even mean by that?," in *Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[15] J. Shi, Z. Yuan, G. Tie, P. Zhou, N. Z. Gong, and L. Sun, "Prompt injection attack to tool selection in LLM agents," in *NDSS*, 2026.

[16] R. Wang, Y. Jia, and N. Z. Gong, "ObliInjection: Order-Oblivious Prompt Injection Attack to LLM Agents with Multi-source Data," in *NDSS*, 2026.

[17] N. V. Pandya, A. Labunets, S. Gao, and E. Fernandes, "May I have your attention? breaking fine-tuning based prompt injection defenses using architecture-aware attacks," *arXiv preprint arXiv:2507.07417*, 2025.

[18] S. Abdelnabi, A. Fay, G. Cherubin, A. Salem, M. Fritz, and A. Paverd, "Get my drift? catching llm task drift with activation deltas," in *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, 2025.

[19] D. Goodin, "New hack uses prompt injection to corrupt Gemini's long-term memory." Ars Technica, Feb. 2025. Accessed: 2026-03-16.

[20] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," 2025. A community-driven standard awareness document identifying the top 10 most critical security risks to Large Language Model (LLM) applications, including prompt injection, sensitive information disclosure, supply chain vulnerabilities, data poisoning, improper output handling, system prompt leakage, vector and embedding vulnerabilities, misinformation, unbounded consumption, and excessive agency. Part of the OWASP GenAI Security Project.

[21] OWASP Agentic Security Initiative, "Agentic AI – Threats and Mitigations," security guide, OWASP Foundation, February 2025. First guide from the OWASP Agentic Security Initiative providing a threat-model-based reference of emerging agentic threats and mitigations. Covers autonomous AI systems enabled by large language models, with structured threat taxonomy covering Agent Design, Agent Memory, Planning & Autonomy, Tool Use, and Deployment & Operations. Expands on the OWASP Top 10 for LLMs and GenAI to address the unique security challenges of agentic AI systems.

[22] National Institute of Standards and Technology, "Artificial intelligence risk management framework: Generative artificial intelligence profile," Tech. Rep. NIST AI 600-1, National Institute of Standards and Technology, July 2024.

[23] NIST, "Guide for conducting risk assessments," Tech. Rep. SP 800-30 Rev. 1, National Institute of Standards and Technology, 2012.

[24] P. Steinberger and OpenClaw Contributors, "OpenClaw: Your own personal AI assistant. any OS. any platform.." `https://github.com/openclaw/openclaw`, 2025. Accessed: 2026-03-16.

[25] B. Schneier, "Attack trees," December 1999. Accessed: 2025.

[26] OWASP GenAI Security Project, "OWASP top 10 for agentic applications for 2026." OWASP Foundation, `https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/`, Dec. 2025. Developed through collaboration with more than 100 industry experts. Accessed: 2026-03-16.

[27] MITRE Corporation, "MITRE ATLAS™: Adversarial threat landscape for artificial-intelligence systems." MITRE Corporation, 2025. Living knowledge base; accessed: 2026-03-16.

[28] Cloud Security Alliance, "Introductory guidance to AICM: The AI controls matrix." Cloud Security Alliance, Nov. 2025. Accessed: 2026-03-16.

[29] Cisco, "Cisco AI defense: Integrated AI security and safety framework." Cisco Systems, Inc., https://www.cisco.com/site/us/en/learn/topics/artificial-intelligence/ai-security-safety-framework.html, 2025. Accessed: 2026-03-16.

[30] A. Zhao, "Why OpenClaw poses new AI-era security risks." https://aaronzhao123.substack.com/p/why-openclaw-poses-new-ai-era-security, 2025. Substack.

[31] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design." https://arxiv.org/abs/2503.18813, 2025.

[32] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Securing ai agents with information-flow control," 2025.

[33] L. Sion, D. Van Landuyt, K. Wuyts, and W. Joosen, "Robust and reusable linddun privacy threat knowledge," *Computers & Security*, vol. 154, p. 104419, 2025.

A Risk Quantification Matrices

Table 1: Capability fit lookup: adversary capability (rows) versus vector skill requirement (columns).

Capability \ Skill Req.	Low	Mod	High
High	High	High	High
Mod	High	High	Mod
Low	High	Mod	Low

Table 2: Combination matrix for deriving overall vector likelihood from capability fit (rows) and residual success likelihood (columns).

Capability \ Res. likelihood	Low	Mod	High
Low	Low	Low	Mod
Mod	Low	Mod	High
High	Mod	High	High

B Threat Source Taxonomy

MATRA adopts the threat source taxonomy from NIST SP 800-30 [23], which distinguishes *adversarial* and *non-adversarial* sources. Adversarial sources are characterised by capability, intent, and targeting; non-adversarial sources are characterised by the range of effects and inherent likelihood of occurrence. Table 4 summarises the taxonomy with examples relevant to agentic AI deployments.

The practitioner selects threat sources from this taxonomy based on credible motivation and a viable access path to the system under review (see Section 2.2). For example, an agentic system exposed to customer email may warrant a *customer* (organisation-level adversarial) source, while a system with autonomous tool execution should always include an *accidental* (non-adversarial) source to account for hallucination-driven impacts.

C OpenClaw: Full Case Study

C.1 Deployment Scenario

We model a realistic deployment in which a solo developer runs a small B2B SaaS product—an online booking platform—and uses OpenClaw on their personal workstation as a productivity tool. The product's PostgreSQL database serves the web application, a background worker for payment processing, and a reporting dashboard; it is the single source of truth for customer data, bookings, and invoices. The developer interacts with the agent primarily via Telegram, issuing day-to-day instructions such as "check

Table 3: Risk Assessment Matrix

		Impact		
		Low	**Moderate**	**High**
Likelihood	**High**	Moderate (3)	High (6)	Very High (9)
	Moderate	Low (2)	Moderate (4)	High (6)
	Low	Very Low (1)	Low (2)	Moderate (3)

Table 4: Threat source taxonomy adapted from NIST SP 800-30 [23].

Type	**Examples**
Adversarial	
Individual	Outsider, insider, privileged insider
Group	Ad hoc, established
Organisation	Competitor, supplier, partner, customer
Nation-State	State-sponsored actor
Non-adversarial	
Accidental	User error, automated process failure

the latest sign-ups," "why is customer X getting errors," or "deploy the fix to staging." An email plugin allows the agent to read and reply to customer support messages on the developer's behalf. The workspace contains the product's source code, deployment scripts, and a `.env` file with production database credentials and third-party API keys.

This deployment is configured as follows:

- **Messaging channels:** Telegram (primary interface for instructing the agent) and email (via community extension plugin, for reading and sending customer support email).
- **LLM provider:** Anthropic Claude, accessed via API key stored in `~/.openclaw/credentials/`.
- **Tools enabled:** `exec` (shell commands), `browser` (Playwright-based web automation), `read`/`write`/`edit` (file I/O), `message` (cross-channel messaging), `memory_search`/`memory_get` (long-term memory).
- **Database access:** The production PostgreSQL database, shared with the web application and background workers; the agent queries it by running `psql` commands via the `exec` tool.
- **Workspace:** `~/.openclaw/workspace/` containing product source code, configuration files, and a `.env` file with database credentials and API keys.
- **Sandboxing:** Off (default configuration).

This scenario is directly grounded in OpenClaw's documented capabilities and default configuration. The addition of email is motivated by its prevalence as a community extension and its role as a high-value attack surface demonstrated in recent security research [30].

C.2 Assets and Impact Scenarios

Following the process in Section 2.1, we identify seven assets from the DFD (Figure 3): API credentials (A1), messaging accounts (A2), workspace files (A3), conversation history and memory (A4), the agentic application (A5), the production PostgreSQL database (A6), and the platform's end users (A7). The database is particularly critical: it is shared with the web application, payment processing worker, and reporting dashboard, making it the single source of truth for customer PII, bookings, and invoices. End users are an asset because the agent can send messages on the developer's behalf—an attacker who

compromises the agent can use this trusted channel to phish customers for their credentials.

Applying the impact assessment process from Section 2.2, we assess each asset against the CIA triad for each threat source. The full impact matrix is provided in Table 6. From this matrix, we derive nine distinct impact scenarios, presented in Table 5. Eight of the nine scenarios are rated High impact, reflecting the deployment's broad attack surface and the sensitivity of the connected assets.

C.3 Threat Source Selection

Following the process in Section 2.2, we select four threat sources from the NIST taxonomy (Appendix B, Table 4) whose capability and access realistically intersect with the deployment's attack surface:

- **Malicious customer** (Organisation → Customer; low capability): exploits the support email channel—a natural, legitimate access path—to inject instructions into the agent's context. This is the most realistic adversarial threat source due to the combination of low barrier, direct access, and clear motivation.

- **Competitor** (Organisation → Competitor; moderate capability): targets the customer database or source code via email or poisoned web pages.

- **Opportunistic outsider** (Individual → Outsider; low–moderate capability): relies on indirect prompt injection via web content the agent encounters during browsing.

- **Accidental** (Non-adversarial; no capability assessment): particularly significant because the agent autonomously executes tool calls without human confirmation. LLM hallucinations can trigger high-impact outcomes—such as corrupted database records—without any malicious input.

C.4 Attack Tree Construction

Following the process in Section 2.3, we construct attack trees in three stages: impacts to objectives, objectives to techniques, and techniques to architecture-specific vectors. Each impact scenario decomposes into two objectives that must both succeed. For adversarial confidentiality impacts, these are *LLM behavioural manipulation* (redirecting the agent's actions) and *data exfiltration* (transmitting data outside the trust boundary). For integrity impacts, the second objective becomes *unauthorized action execution*—reflecting OpenClaw's broad write access to shell, files, messaging, and database. For availability, it becomes *excessive resource consumption*. Non-adversarial paths mirror this structure, with *unexpected LLM behaviour* (hallucination) replacing deliberate manipulation.

Two architectural features shape the resulting trees. First, OpenClaw supports two categories of exfiltration techniques: *tool-based exfiltration*, where the agent explicitly invokes a tool to transmit data (e.g., `exec: curl`, `message: email`, Figure 9), and *rendered-markup exfiltration*, where the agent embeds queried data in a URL within its Telegram response and the client auto-fetches the resource (Figure 10). A critical distinction for sandbox effectiveness is that only `exec`-based tools run inside the Docker container; the message tool and browser run on the gateway host, meaning their exfiltration vectors survive container-level network isolation. Second, because Telegram and email are accessible to external parties, *direct* prompt injection (Figure 7) is a primary concern—unlike systems where the only input channel is an authenticated internal chat. Indirect prompt injection vectors are particularly numerous due to the agent's broad input surface (Figure 8).

Not all vectors apply to all scenarios; only architecturally viable combinations are included in each tree. For example, exfiltration vectors do not appear in integrity trees, and expensive query vectors apply exclusively to the database availability scenario. This scoping ensures each tree is realistic and prevents double-counting in aggregate risk assessment. The full trees are provided in Appendix C.7.

C.5 Data Flow Diagrams

Figure 3 presents the high-level data flow diagram for the OpenClaw deployment, showing processes, data stores, external actors, and trust boundaries.

C.6 Business Impact Assessment Matrix

Table 6 presents the stakeholder-assessed business impact for each asset and CIA dimension in the OpenClaw deployment. For availability, the impact is independent of the threat source, so a single rating is assigned. Cells marked "—" indicate combinations that are not applicable to this deployment. The matrix assesses impact only; the specific threat source's capability is introduced during attack tree construction (Section 2.3), where it determines the capability fit for each vector. Not every threat source is assessed against every impact scenario: if a threat source lacks credible intent or a viable targeting path for a given scenario, that combination is excluded from analysis (see Section 2.2).

C.7 Attack Tree Construction Details

This appendix presents the full attack tree decomposition for the OpenClaw deployment, constructed in three stages following the process in Section 2.3. The decomposition proceeds from impacts to objectives (Figures 4 and 5), objectives to techniques (Figure 6), and techniques to architecture-specific vectors (Figures 7–13).

C.7.1 From Impacts to Objectives

Figures 4 and 5 show the decomposition of adversarial and non-adversarial impact scenarios into their constituent objectives.

The integrity objective reflects OpenClaw's broad write access. In read-only architectures, integrity impacts are limited to manipulating the text returned to the user. OpenClaw's single agent has full write access, i.e., shell execution, file I/O, cross-channel messaging, and persistent memory, so integrity impacts extend to system state changes such as modifying files, sending messages as the user, corrupting database records, or poisoning the agent's long-term memory.

In OpenClaw's single-agent architecture, the non-adversarial path deserves particular attention. Because the agent autonomously executes tool calls without human confirmation, the bottleneck for non-adversarial impacts shifts from action execution (which happens automatically) to the hallucination probability itself. The two-objective decomposition correctly captures that both the hallucination *and* the harmful action must co-occur for an impact to materialise.

C.7.2 From Objectives to Techniques

Figure 6 maps each objective to the attack techniques applicable in the OpenClaw architecture.

C.7.3 From Techniques to Vectors

Figures 7–13 enumerate the concrete attack vectors for each technique, grounded in the OpenClaw architecture and its DFD (Figure 3).

C.8 Deep Dive: Database Risks—Availability and Integrity (IS5, IS8)

This deep dive examines two database-related impact scenarios that arise from the agent's unrestricted SQL access via the `exec` tool (see Figures 14 and 15 for the full attack trees and Table 7 for the risk scores). Because the production database is shared with the booking platform's web application, payment processing worker, and reporting dashboard, any disruption or corruption directly affects paying customers.

We present IS5 (availability) as a primarily *adversarial* concern and IS8 (integrity) as a realistic *non-adversarial* risk. This pairing illustrates how the same asset can be threatened through fundamentally different paths, each requiring different controls.

Table 5: Impact scenarios for the OpenClaw deployment, derived from the business impact assessment (Appendix C.6).

ID	Asset	CIA	Impact Scenario	Business Impact
IS1	A1 Credentials	C	API keys exfiltrated	High
IS2	A2 Messaging	I	Unauthorized messages sent to customers or contacts	High
IS3	A3 Workspace	C	Source code / secrets leaked	High
IS4	A4 Memory	I	Long-term memory poisoned	Moderate
IS5	A6 Database	A	Booking platform and payments disrupted (heavy queries)	High
IS6	A6 Database	C	Customer PII and booking records exfiltrated	High
IS7	A5 Agentic App	I	Security controls weakened via config change	High
IS8	A6 Database	I	Booking or invoice records corrupted	High
IS9	A7 End Users	C/I	Customers phished via agent-sent messages	High

Table 6: Business impact assessment matrix for the OpenClaw deployment. The adversarial column covers all adversarial threat sources, as the business impact of a given compromise is independent of the attacker's identity.

Asset	CIA	Adversarial	Accidental	All (Avail.)
A1 Credentials	C	High	Low	—
	I	—	—	—
	A	—	—	—
A2 Messaging	C	—	—	—
	I	High	Moderate	—
	A	—	—	—
A3 Workspace	C	High	—	—
	I	High	Moderate	—
	A	—	—	—
A4 Memory	C	—	—	—
	I	Moderate	Moderate	—
	A	—	—	—
A5 Agentic App	C	—	—	—
	I	High	Moderate	—
	A	—	—	Moderate
A6 Database	C	High	—	—
	I	High	Moderate	—
	A	—	—	High
A7 End Users	C	High	—	—
	I	High	—	—
	A	—	—	—

C.8.1 IS5: Booking Platform Disrupted (Adversarial)

Objectives. Two objectives must succeed: *LLM behavioural manipulation* (to direct the agent to issue a specific query) and *excessive resource consumption* (the query must be expensive enough to degrade availability for all connected services).

Narrative. An attacker places a crafted instruction in a web page that the developer asks the agent to research. The injected instruction directs the agent to execute a query such as `SELECT * FROM bookings a CROSS JOIN bookings b` against the production database via the `exec` tool (see Figure 12 for the full set of expensive query vectors). This cartesian join produces an exponentially large result set, consuming all available database connections and CPU, taking down the booking platform for all customers.

C.8.2 IS8: Booking Records Corrupted (Non-adversarial)

Objectives. Two objectives must co-occur: *unexpected LLM behaviour* (the agent generating an incorrect write query) and *unauthorized action execution* (the query must actually modify data). A hallucinated but syntactically invalid query, or one that is read-only, does not cause an integrity impact; both conditions must be met.

Narrative. The developer asks the agent to "mark all unpaid bookings older than 90 days as cancelled." The agent generates a syntactically valid but semantically incorrect query (see Figure 13)—for example, `UPDATE bookings SET status = 'cancelled' WHERE created_at < now() - interval '90 days'` omitting the `AND payment_status = 'unpaid'` predicate. The query cancels all bookings older than 90 days, including paid and confirmed ones. Customers receive erroneous cancellation notifications, the payment worker attempts refunds for already-completed transactions, and the reporting dashboard shows incorrect revenue figures. No malicious input is required; the impact arises from the LLM's imperfect understanding of the business logic encoded in the database schema.

C.8.3 Full Attack Trees with Likelihood Scores

C.8.4 Risk Assessment

In the default configuration, IS5 reaches Very High risk (9): both the injection and resource consumption objectives are High, and the shared production database drives impact to High. IS8 reaches High risk (6) with Moderate likelihood, bottlenecked by the hallucination objective (Moderate inherent probability), while the action execution objective is High because the agent has unrestricted write access.

Despite arriving at different default risk levels, both scenarios benefit significantly from targeted controls, each reducing to Moderate (3). The two scenarios require different controls because they are bottlenecked by different objectives.

C.8.5 Control Effect Analysis

IS5 and IS8 require *different* database-level controls, illustrating that the same asset can demand distinct mitigations depending on the CIA dimension under threat.

IS5 (Availability).

- **Query timeout:** A PostgreSQL `statement_timeout` caps resource consumption per query.
- **Query validation:** Running `EXPLAIN` on generated queries before execution and rejecting those above a cost threshold.
- **Connection limiting:** A PostgreSQL `max_connections` setting or connection pool limit prevents concurrent session exhaustion.

IS8 (Integrity).

- **Read-only database user:** A read-only PostgreSQL role eliminates `UPDATE`, `DELETE`, and `INSERT`, reducing the action execution objective from High to Low.

Table 7: Risk scores for IS5 and IS8 under different control configurations.

Scenario	Threat Source	Configuration	Likelihood	Impact	Risk
IS5	Competitor	Default	High	High	Very High (9)
IS5	Competitor	+ query guardrails	Low	High	Moderate (3)
IS8	Accidental	Default	Moderate	High	High (6)
IS8	Accidental	+ read-only DB role	Low	High	Moderate (3)

- **Transaction review:** Wrapping write queries in `BEGIN`/`ROLLBACK` with human confirmation before `COMMIT`.

Notably, **Docker sandboxing does not help** with either scenario; the database must remain accessible for the tool to function. IS6 (confidentiality) presents a parallel challenge: while sandboxing blocks `exec`-based exfiltration, message-based and rendered-markup vectors survive because they run outside the container. All three scenarios illustrate that effective controls must operate at the layer where the threat materialises, i.e., the database layer for IS5 and IS8, and the messaging and output layer for IS6.

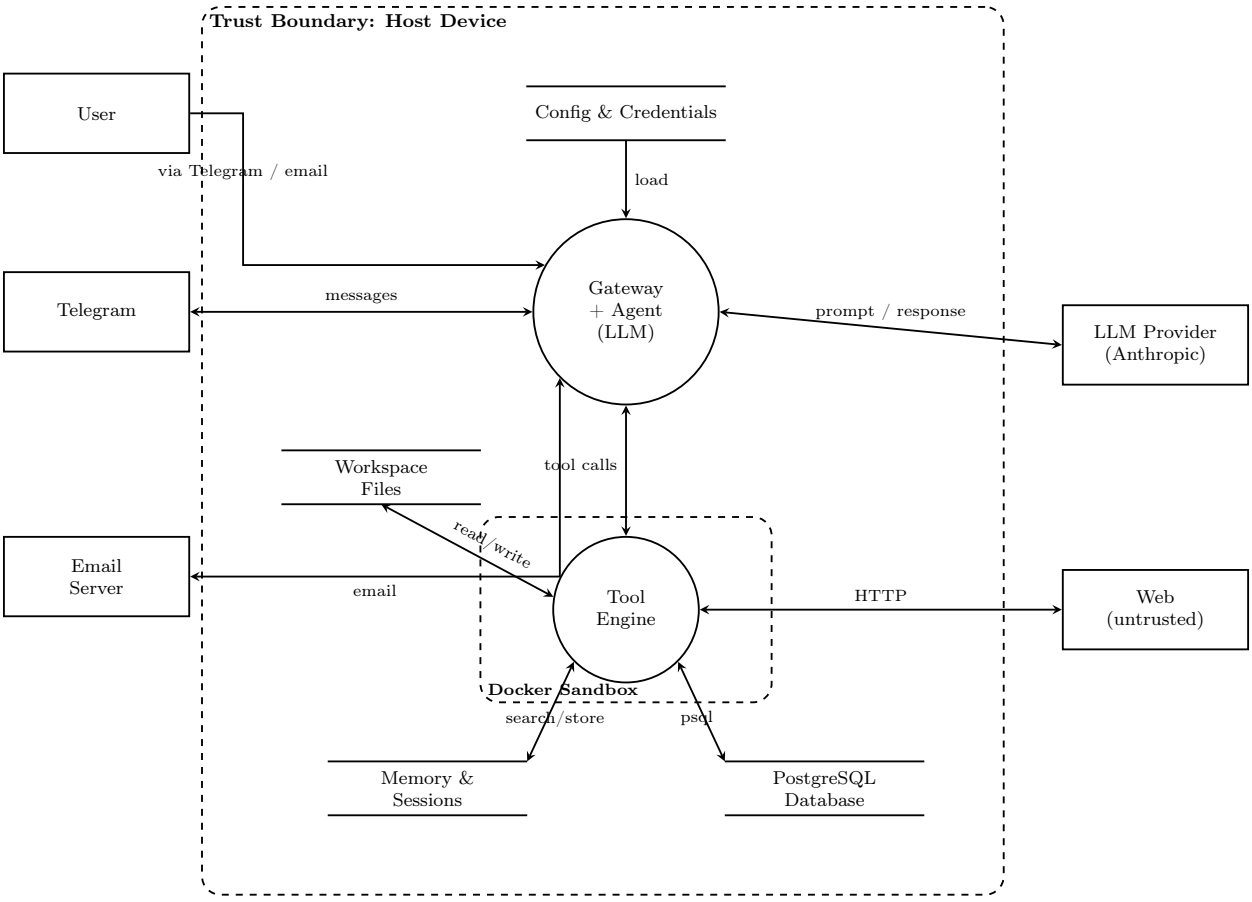


Figure 3: High-level DFD of the OpenClaw deployment scenario. The gateway process mediates all interactions between messaging channels, the LLM provider, and the tool engine. Bidirectional arrows on Telegram and email reflect that the gateway both receives inbound messages and sends outbound messages (the message tool runs on the gateway, outside the Docker sandbox). Only the tool engine (exec, file I/O) runs inside the optional sandbox container.

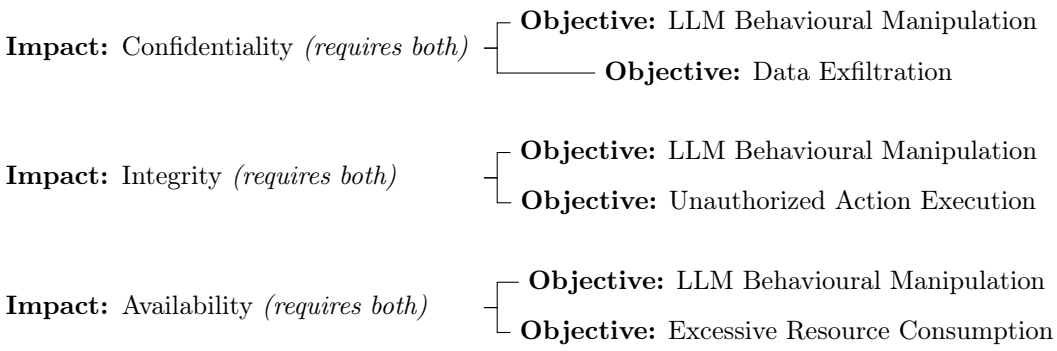


Figure 4: Adversarial: Impact → Objectives (both required).

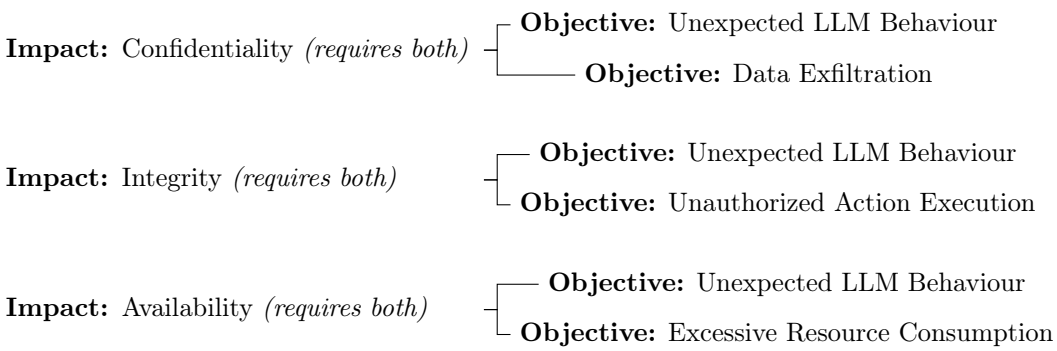


Figure 5: Non-adversarial: Impact → Objectives (both required). The decomposition mirrors the adversarial case, with "Unexpected LLM Behaviour" replacing "LLM Behavioural Manipulation" as the first objective.



Figure 6: Objectives → Techniques in the OpenClaw architecture.

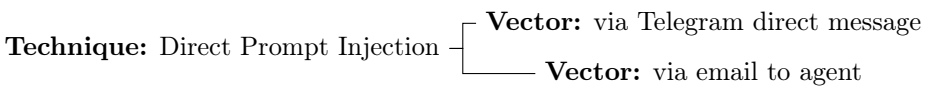


Figure 7: Attack vectors for *Direct Prompt Injection* in OpenClaw.

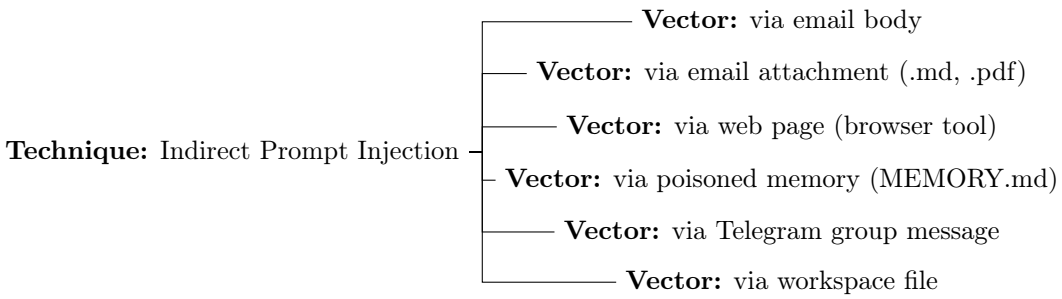


Figure 8: Attack vectors for *Indirect Prompt Injection* in OpenClaw.

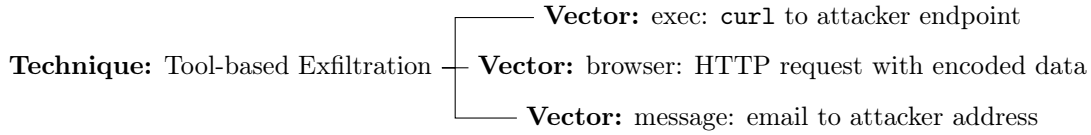


Figure 9: Attack vectors for *Tool-based Exfiltration* in OpenClaw.

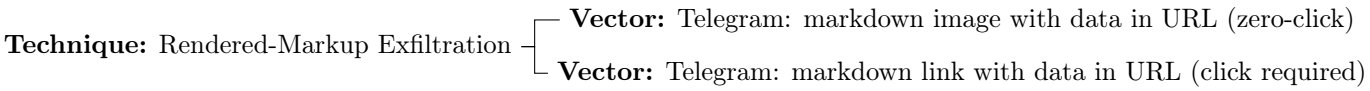


Figure 10: Attack vectors for *Rendered-Markup Exfiltration* in OpenClaw. The agent encodes queried data in a URL embedded in its Telegram response. Telegram auto-fetches image URLs (zero-click); links require user interaction.

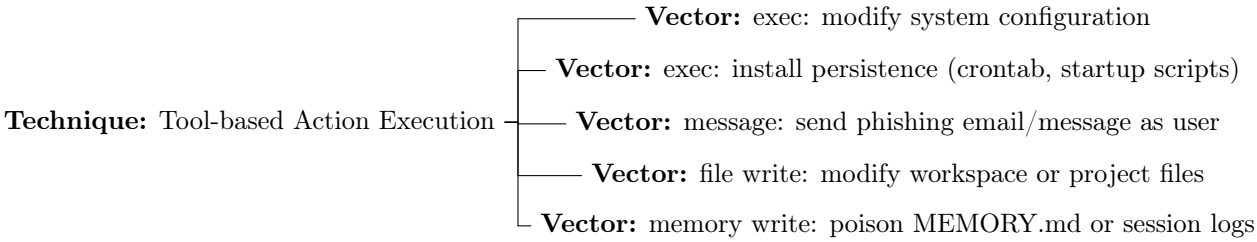


Figure 11: Attack vectors for *Tool-based Action Execution* in OpenClaw.

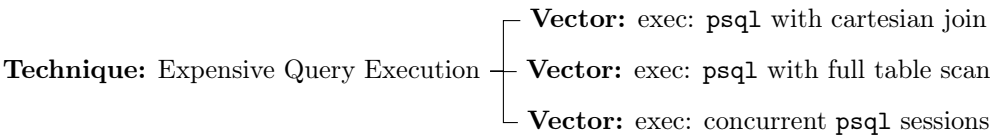


Figure 12: Attack vectors for *Expensive Query Execution* targeting the PostgreSQL database.

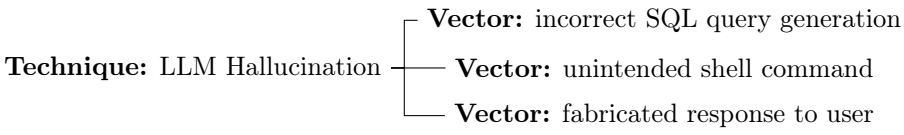


Figure 13: Attack vectors for *LLM Hallucination* in OpenClaw.

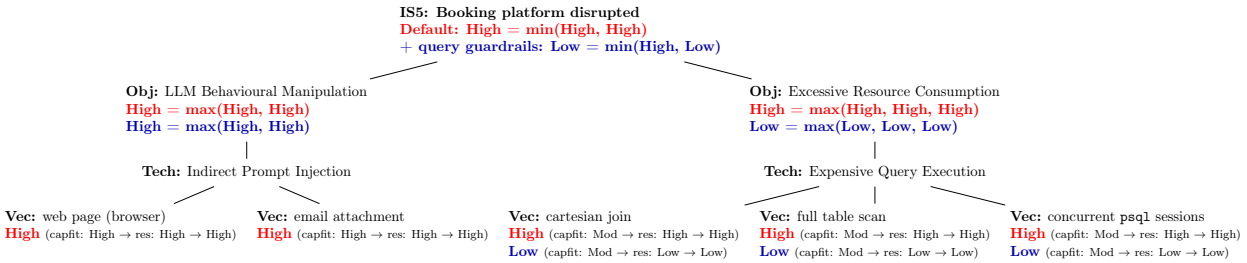


Figure 14: Attack tree for IS5 (database availability, adversarial path), assessed for threat source: *competitor* (capability: Moderate). Vector labels follow the format *capfit: capability fit → res: residual success likelihood → combined likelihood* (see Tables 1 and 2). Query guardrails reduce the resource consumption objective from High to Low: `statement_timeout` and `EXPLAIN` cost validation cap query execution (res: Low for cartesian join and full table scan), while `max_connections` limits prevent concurrent session exhaustion (res: Low for psql sessions). Injection vectors have no applicable controls (res: High).

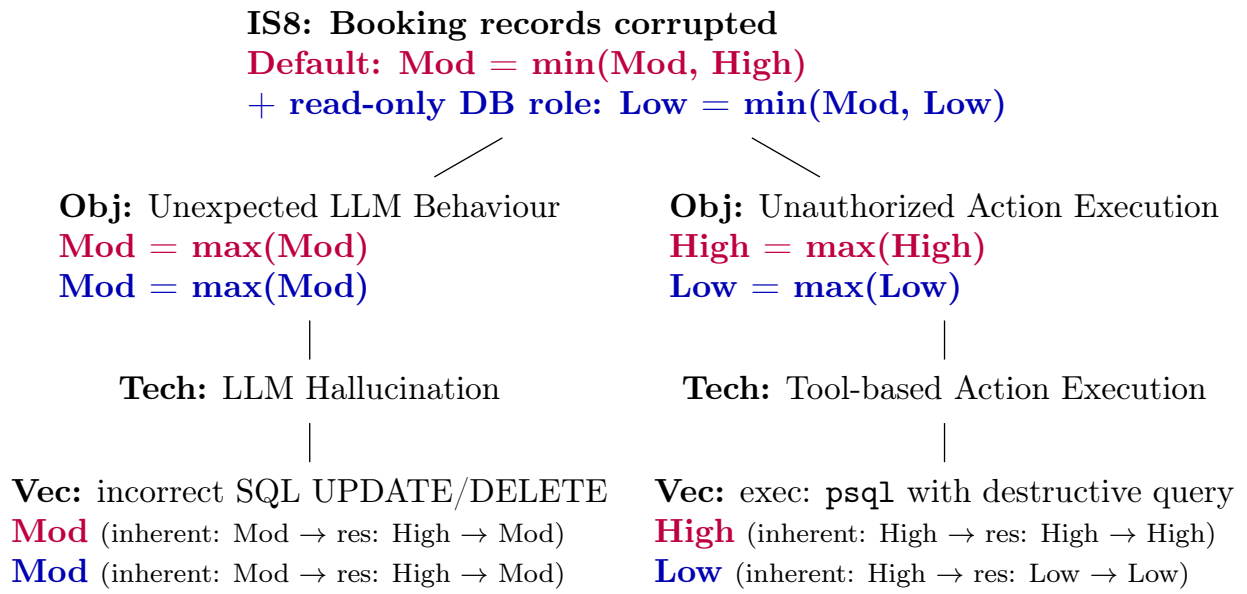


Figure 15: Attack tree for IS8 (database integrity, non-adversarial path), assessed for threat source: *accidental* (non-adversarial). Vector labels follow the format *inherent: manifestation likelihood → res: residual success likelihood → resulting likelihood*. For non-adversarial threats, inherent likelihood replaces capability fit; the inherent probability bounds the resulting likelihood regardless of residual success. Hallucination has an inherent Moderate probability; no control reduces it (res: High), but the result remains Moderate. In the default configuration, the agent has unrestricted database write access (res: High). A read-only database role blocks all write operations (res: Low), reducing the action execution objective from High to Low.